

Ryan Ramdihal

EEL 4742

Lab 9:Serial Peripheral Interface (SPI) & LCD Pixel Display

Introduction:

In this lab, we will use the Serial Peripheral Interface (SPI) to communicate data to the pixel display on the BoosterPack board. The SPI uses two data wires. Our first objective is to configure the SPI's port pins and their eUSCI for SPI operation, given by the two function headers. Followed by changing the background, foreground, drawing shapes, with 3 different colors, using the Graphics library provided.

9.1:Serial Peripheral Interface (SPI)

The first objective is to look at figure 9.1 in the lab manual for the ports/bits that the SPI pins booster pack uses. After configuring the serial clock and the SIMO, looking on the LCD Display side of the figure we match the function to the correct ports in the port function. Next, is to configure the spiinit function. By following the instructions given in the comments of the function to configure the fields to, we look into the User's guide for the correct syntax for each configuration.

9.1 Functions Code:

```
void HAL_LCD_PortInit(void)
{
    ///////////////////////////////////
    // Configuring the SPI pins
    ///////////////////////////////////

    // Configure UCB0CLK/P1.4 pin to serial clock
    // P1DIR = DNC, P1SEL1 = 0, P1SEL0 = 1
    P1SEL1 &= ~BIT4;
    P1SEL0 |= BIT4;
    // Configure UCB0SIMO/P1.6 pin to SIMO
    // P1DIR = DNC, P1SEL1 = 0, P1SEL0 = 1
    P1SEL1 &= ~BIT6;
    P1SEL0 |= BIT6;
    // OK to ignore UCB0STE/P1.5 since we'll connect the display's enable bit to low (enabled all the time)
    // OK to ignore UCB0SOMI/P1.7 since the display doesn't give back any data

    //found in lab manual
    ///////////////////////////////////
    // Configuring the display's other pins
    ///////////////////////////////////
    // Set reset pin as output.
    P9DIR |= BIT4; // Pin 9.4
    // Set the data/command pin as output.
    P2DIR |= BIT3; //Pin 2.3
    // Set the chip select pin as output.
```

```

P2DIR |= BIT5; //Pin 2.5
return;
}
void HAL_LCD_Spilnit(void)
{
    ///////////////////////////////////
    // SPI configuration
    ///////////////////////////////////
    // Put eUSCI in reset state and set all fields in the register to 0
    UCB0CTLW0 = UCSWRST;
    // Fields that need to be nonzero are changed below
    //table found on page 816 of users guide
    // Set clock phase to "capture on 1st edge, change on following edge"
    UCB0CTLW0 |= UCCKPH;
    // Set clock polarity to "inactive low"
    UCB0CTLW0 &= ~UCCKPL;
    // Set data order to "transmit MSB first"
    UCB0CTLW0 |= UCMSB;
    // Set data size to 8-bit
    UCB0CTLW0 &= ~UC7BIT;
    // Set MCU to "SPI master"
    UCB0CTLW0 |= UCMST;
    // Set SPI to "3-pin SPI" (we won't use eUSCI's chip select)
    UCB0CTLW0 &= ~UCMODE_3;
    // Set module to synchronous mode
    UCB0CTLW0 |= UCSYNC;
    // Set clock to SMCLK
    UCB0CTLW0 |= UCSSEL_3;
    // Configure the clock divider (SMCLK is from DCO at 8 MHz; run SPI at 8 MHz using SMCLK)
    // Divide by 1
    UCB0BRW = 1;
    // Exit the reset state at the end of the configuration
    UCB0CTLW0 &= ~UCSWRST;
    //in lab manual
    // Set CS' (chip select) bit to 0 (display always enabled)
    P2OUT &= ~BIT5;
    // Set DC' bit to 0 (assume data)
    P2OUT &= ~BIT3;
    /*/
    return;
}

```

9.2:Using the Graphics Library

Conclusion:

This lab introduces the utilization of the Serial Peripheral Interface (SPI) to facilitate communication with the pixel display on the BoosterPack board. The initial focus is on configuring the SPI port pins and their associated eUSCI for SPI operation, as outlined by the provided function headers. Subsequently, we explored the functionalities of the Graphics library by modifying background and foreground colors and drawing shapes in three different colors.

Student Q&A

1. Is SPI implemented as simplex or full-duplex in this experiment?

full-duplex

2. What SPI clock frequency did we set up in this lab?

8 MHz

3. What I2C clock frequency did we set up in this lab?

1MHz

4. What is the maximum SPI clock frequency that is supported by the eUSCI module? Look in the microcontroller's data sheet in Table 5-18.

16MHz

5. Show how you computed the I2C clock divider in the last part.

$1\text{MHz} / (2 \times 320\text{KHz}) = 1.5625$ divider